

# Data Engineering for Heterogeneous Multi-Sensor Fusion Platforms

**A monograph on canonical contracts, streaming reliability, geospatial state, and replayable decision-support for counter-swarm defence**

---

**Author:** Victor.I

**Programme:** Counter-Swarm Defence research monographs (Data Engineering)

**Audience:** Data engineers, systems architects, fusion/tracking leads, security reviewers, and technical programme managers

**Scope:** Defensive decision-support software only — sensing, normalisation, streaming, storage, lineage, quality, retention, and replay. This document does not prescribe weapon fire control, kinetic effects, or electronic-attack execution.

**Normative platform artefacts referenced:**

- `schemas/observation.v1.json` — canonical observation contract
  - `architecture/data-flow.md` — event families, retention drafts, contradiction handling
  - `architecture/interfaces.md` — ICD-01 through ICD-15
  - `docs/data/architecture.md` — DE ownership summary
  - `architecture/tradeoffs.md` — bus and store technology decisions
- 

## Abstract

---

Heterogeneous multi-sensor platforms fail less often from a shortage of algorithms than from a shortage of **honest data contracts**. Radar, electro-optical (EO), infrared (IR), radio-frequency (RF), acoustic, and telemetry sources arrive at incompatible rates, with different clocks, coordinate reference systems (CRS), uncertainty semantics, and failure modes. In a counter-swarm defensive decision-support system, those streams must be normalised into a stable observation contract, transported on a durable event bus, projected into geospatial operational state, and retained in forms that permit audit, replay, and model evaluation — without encoding business risk policy inside ingest.

This monograph develops a production-grade data-engineering design for that problem space, mapped explicitly to Counter-Swarm Defence artefacts: `observation.v1`, the Kafka-API event bus, the quarantine path (`observation.quarantine.v1`), PostgreSQL/PostGIS hot state, object-store cold artefacts, and lineage from raw capture through fused tracks to human decisions. Drawing on established systems literature — including Kleppmann’s treatment of unreliable clocks, schemas, and logs; Kreps’ log-centric streaming thesis; the Dataflow model of watermarks and late data; and geospatial standards embodied in PostGIS/OGC practice — it specifies how to handle late, missing, duplicate, and contradictory observations; how to evolve schemas without breaking consumers; how to prove contracts with tests; and how to build the platform incrementally without premature multi-region complexity.

The central claim is operational rather than rhetorical: **fusion quality is bounded by ingest honesty**. If timestamps lie, if CRS is implicit, if duplicates are silent, if quarantine is optional, and if replay cannot reconstruct evidence links, then downstream tracking, behaviour indication, and risk assessment inherit irreducible epistemic debt. Data engineering’s job is to make that debt visible, measurable, and reversible.

**Keywords:** multi-sensor fusion, canonical schema, event streaming, PostGIS, time synchronisation, data quality, lineage, schema evolution, replay, defensive C2 decision-support

---

## Table of contents

---

1. Introduction and problem restatement
2. Assumptions, ambiguities, and success conditions
3. Related work and intellectual lineage
4. Counter-Swarm data path as the object of study
5. Canonical schemas and the `observation.v1` contract
6. Heterogeneous failure: late, missing, duplicate, contradictory
7. Event streaming: logs, partitions, delivery, and quarantine
8. Geospatial hot state with PostgreSQL and PostGIS
9. Time synchronisation, ordering, and watermarks
10. Lineage, provenance, and evidence graphs
11. Data quality programmes and quarantine semantics
12. Retention, tiering, and cost governance
13. Replay, reprocessing, and after-action reconstruction
14. Schema evolution and registry policy
15. Architecture alternatives and justified recommendation
16. Failure modes and operational burden
17. Proof: contract tests, property tests, and evaluation harnesses
18. Build guide (incremental delivery)
19. Pre-ship gate and open decisions
20. Conclusion
21. References
22. Appendices A–AF (mapping, quarantine examples, deep dives, worked path, SLOs, runbooks, open questions)

---

# 1. Introduction and problem restatement

---

## 1.1 The engineering problem in plain language

A counter-swarm defensive decision-support platform must continuously answer operator-facing questions of the form: *What is moving where? With what confidence? Based on which sensors? What changed since the last assessment? Can we reconstruct why the system recommended a category?* Those questions are not primarily model questions. They are **data-system questions** about representation, ordering, durability, identity, and provenance.

The physical world presents sensors that:

- sample at rates spanning Hz to kHz (and bursts of frames for EO/IR);
- stamp time with clocks that drift, jump, or freeze;
- report position in different CRS or in sensor-local frames;
- omit fields under load or when calibration fails;
- retransmit the same detection after network recovery;
- contradict each other about class, kinematics, or existence.

A naive “ingest everything into a database and fuse later” design collapses under these conditions. A naive “stream everything and hope consumers are careful” design externalises the same collapse to every subscriber. The durable approach — the one this monograph defends — is:

1. Force a **canonical observation** at the system boundary.
2. Transport it on a **replayable log** with explicit quarantine.
3. Project **operational geospatial state** into a hot store designed for operator queries.
4. Keep **raw and derived artefacts** addressable for lineage.
5. Treat **schema change** as a controlled industrial process, not an ad-hoc edit.

## 1.2 Restatement for Counter-Swarm Defence

Counter-Swarm Defence (this repository) is an event-driven modular system: adapters produce candidate observations; a normaliser validates and enriches; a Kafka-compatible bus carries `observation.v1` and derived event families; detection, tracking, fusion, behaviour, and risk consume those events; operators approve human decisions; audit retains material history. Data Engineering owns canonical schemas, the bus, hot/warm/cold stores, lineage, quality, retention, and replay — and must **not** encode business risk policy inside raw ingest (`docs/systems/agent-responsibility-matrix.md`).

The problem this monograph solves is therefore:

Design and justify the data platform that makes Counter-Swarm’s fusion and decision-support chain *honest under adversarial and messy sensor conditions*, while remaining operable by a small skilled team for years.

### 1.3 What “research-grade” means here

A dissertation typically contributes novel theory or empirics over years. This monograph is **research-structured**: problem framing, literature grounding, architecture exploration with trade-offs, failure analysis, evaluation/proof methods, and a build programme mapped to a concrete system. It is not a claim of unpublished experimental novelty. Where the literature is thin relative to defence fusion practice, the text marks uncertainty rather than inventing citations.

### 1.4 Defensive scope boundary

All designs assume **human-authorised response categories** (e.g., heighten monitoring, cue sensor, notify external). The integration surface forbids weapon aimpoints and related effector parameters (architecture/interfaces.md ICD-11). Data engineering still matters acutely: bad evidence graphs produce bad recommendations, and bad audit trails make governance theatre.

---

## 2. Assumptions, ambiguities, and success conditions

---

### 2.1 Stated assumptions

| ID | Assumption                                                                                    | Why it might be wrong                                        | Cheap validation                                       |
|----|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------|
| A1 | Sensors can be adapted to emit candidate observations conforming to a shared schema family    | Vendor SDKs may be closed, rate-extreme, or binary-only      | Spike one adapter per modality; measure transform cost |
| A2 | A Kafka-API bus is acceptable operationally (Kafka or Redpanda)                               | Some deployments forbid JVM ops or demand cloud pub/sub only | Customer constraint matrix; lab soak test              |
| A3 | Postgres+PostGIS can hold hot operational state for MVP volumes                               | Extreme track/obs history may exceed comfortable write rates | Synthetic load with site-scale scenarios               |
| A4 | Bounded out-of-order arrival (seconds to low minutes) is the common case                      | Denied/degraded networks can produce hours of delay          | Fault-injection in sim (ICD-15)                        |
| A5 | Operators need geospatial query + evidence drill-down more than analytical SQL on raw streams | Analysts may demand warehouse-first                          | Interview + after-action workflows                     |
| A6 | Schema registry BACKWARD compatibility within a major version is enforceable                  | Teams bypass registries under pressure                       | CI gates; produce-path rejection                       |

If assumptions outnumber facts for a given deployment, Stage 0 must instrument the sim and one live-like sensor path before freezing storage topology.

### 2.2 Ambiguities that must not be papered over

1. **Avro vs JSON Schema on the bus.** ICD-02 still marks “Avro/JSON Schema — TBD”. JSON Schema already exists in schemas/observation.v1.json. The monograph recommends JSON Schema (or JSON Schema + binary encoding later) for Stage 0–1

readability and contract tests, with an escape hatch to Avro/Protobuf if profiling demands — without changing *logical* field semantics.

2. **Partition key:** `sensor_id` vs `site_id`. Per-sensor ordering aids idempotency and monotonicity checks; per-site ordering aids local fusion locality. Recommendation: partition by `site_id` with `sensor_id` in the key suffix or headers for consumer-side grouping, unless a site has extreme multi-sensor fan-in requiring sensor partitions.
3. **Quarantine as topic vs DLQ vs DB table.** Logical event `observation.quarantine.v1` can be a bus topic (preferred for uniformity) with optional projection to a quarantine table for operator tooling.
4. **“Warm” tier.** Docs mention hot/warm/cold; warm is underspecified. Treat warm as *compressed recent history in object storage + selective PostGIS partitions* until volume justifies a dedicated TSDB.

## 2.3 What must be true for success

1. Every operational observation on the happy path validates against a versioned schema.
2. Invalid or suspicious candidates are quarantined with reason codes — never silently dropped without metrics.
3. Downstream services can ignore unknown fields (forward compatibility) and can run against N and N–1 (deployment overlap).
4. A time-range replay can reconstruct tracks and evidence links for an incident under test.
5. Lineage from `human.decision.v1` back to contributing `observation_ids` is queryable.
6. Risk policy does not live in the normaliser.

---

## 3. Related work and intellectual lineage

### 3.1 Data-intensive systems foundations

Martin Kleppmann’s *Designing Data-Intensive Applications* (O’Reilly, 2017) is the primary systems text for this monograph’s vocabulary: unreliable clocks, batch vs stream, encoding/evolution, logs as truth, and the distinction between databases and derived views. Counter-Swarm’s event-driven core is essentially Kleppmann’s “turning the database inside out” applied to a fusion pipeline: the log is the system of record for *facts about sensing*; PostGIS holds *projections* optimised for operator query.

Jay Kreps’ work on Kafka — including the original LinkedIn engineering paper with Narkhede and Rao (Kreps, Narkhede & Rao, 2011) and the essay *The Log: What every software engineer should know about real-time data’s unifying abstraction* (Kreps, 2013) — supplies the operational thesis: a durable, partitioned, append-only commit log enables fan-out, replay, and decoupling. Neha Narkhede, Gwen Shapira, and Todd Palino’s *Kafka: The Definitive Guide* (O’Reilly) remains the practical companion for producer idempotence, consumer groups, and retention.

### 3.2 Stream processing semantics

Tyler Akidau and colleagues’ Dataflow model (Akidau et al., 2015, VLDB) formalises **event time vs processing time**, **watermarks**, and **allowed lateness** — concepts Counter-Swarm must adopt even if the first implementation is “consumer-side watermarks in tracking/fusion” rather than a full Beam/Flink deployment. Apache Flink’s public model (Carbone et al., 2015) similarly treats event-time windows and late arrivals as first-class.

Exactly-once rhetoric must be handled carefully. In practice, Counter-Swarm targets **effectively-once business outcomes** via idempotent keys and transactional outboxes where needed (especially approval/integration), while the sensor bus remains **at-least-once** with dedupe — matching Kleppmann’s caution that “exactly-once” is end-to-end and application-defined.

### 3.3 Multi-sensor fusion context (data implications)

The JDL/DFIG data fusion model tradition (e.g., Steinberg, Bowman & White; Hall & Llinas surveys) separates levels of fusion from raw measurements to situation assessment. Data engineering does not replace association algorithms (Bar-Shalom’s multitarget tracking lineage; Blackman & Popoli), but it **constrains** them: without stable IDs, timebases, and uncertainty fields, statistical association becomes storytelling.

Waltz and Llinas’ *Multisensor Data Fusion* and the *Handbook of Multisensor Data Fusion* (Liggins, Hall & Llinas) emphasise registration, alignment, and pedigree — pedigree is lineage by another name. This monograph treats pedigree as a platform responsibility, not an optional ML metadata field.

### 3.4 Geospatial and temporal storage

PostGIS extends PostgreSQL with OGC-aligned geometry/geography types and spatial indexes (GiST/SP-GiST), making it a pragmatic hot store for tracks, geofences, and incident footprints. OGC Simple Features and CRS registry practice (EPSG codes; WGS 84 as EPSG:4326) are the interoperability baseline. For high-rate time-series pressure, TimescaleDB’s hypertables (as a PostgreSQL extension) are a documented escape hatch — consistent with Counter-Swarm’s trade-off note: start with Postgres+PostGIS; add Timescale if write pressure demands (architecture/tradeoffs.md).

RFC 3339 timestamps and a single platform convention of UTC storage follow internet and ops practice; sensor-native time remains in `timestamp_utc` after conversion, with quality flags when conversion is lossy.

### 3.5 Time synchronisation

NTP (Mills’ long-running IETF work) and IEEE 1588 Precision Time Protocol (PTP) define practical synchronisation for enterprise and industrial systems. Defence sites may mix GNSS-disciplined oscillators at sensors with NTP at servers. Spanner’s TrueTime (Corbett et al., 2012) is instructive as an existence proof that **uncertainty intervals** beat pretend precision — Counter-Swarm should store skew estimates in `quality` rather than inventing false microsecond agreement across modalities.

Leslie Lamport’s “Time, Clocks, and the Ordering of Events in a Distributed System” (1978) remains the conceptual warning: without a shared timeline, “happened-before” across sensors is a logical fiction unless the platform defines ordering rules.

### 3.6 Quality, lineage, and warehouse practice

Data quality frameworks in industry (completeness, consistency, timeliness, accuracy, uniqueness) map cleanly onto quarantine reason codes. Lineage systems (OpenLineage as a contemporary open standard; classical data-warehouse lineage in Kimball-style practice) motivate persisting model versions and adapter versions on derived events — already anticipated in Counter-Swarm’s detection and risk event notes.

Nathan Marz and James Warren’s *Big Data* popularised Lambda architecture (batch + speed layers). Counter-Swarm should **not** blindly adopt dual pipelines; a Kappa-style log-centric design (replay from Kafka/object store) is closer to Kreps and to the platform’s ADR direction — with batch exports for DS evaluation as a derived path, not a second source of truth.

### 3.7 What this monograph does not cite

No fabricated DOIs, no invented “2024 arXiv” papers for rhetorical weight, no faux STANAG clause numbers. Where defence-specific ICDs are customer-private, the text uses Counter-Swarm’s public-in-repo contracts only.

---

## 4. Counter-Swarm data path as the object of study

---

### 4.1 End-to-end flow (normative sketch)

```
Physical sensors / Digital twin
  → sensor-adapter-*   (vendor/sim → candidate observation)
  → normaliser         (validate, CRS, time, idempotency, quality)
  → event bus          (observation.v1 | observation.quarantine.v1)
  → detection / tracking / quality consumers
  → fusion → behaviour → risk → alert → decision support
  → human approval → audit (+ optional integration handoff)
```

Storage fan-out from the bus:

- Hot DB projections (tracks, alerts, incidents, decisions) — Postgres+PostGIS
- Audit append log
- Object store (raw payloads, clips, large artefacts)
- Analytics export (batch) for evaluation

### 4.2 Event families (logical)

From architecture/data-flow.md:

| Event                     | Role                                            |
|---------------------------|-------------------------------------------------|
| observation.v1            | Canonical sensor fact                           |
| observation.quarantine.v1 | Invalid/suspicious candidates                   |
| detection.v1              | Detector outputs + model version                |
| track.update.v1           | State estimate, covariance, evidence obs IDs    |
| behaviour.indicator.v1    | Coordination/anomaly indicators (not decisions) |
| risk.assessment.v1        | Graded risk + missing-info                      |
| alert.v1                  | Policy-tied prioritisation                      |
| recommendation.v1         | Category suggestion                             |
| human.decision.v1         | Authoritative human action category             |
| integration.handoff.v1    | External notification after decision            |
| sensor.health.v1          | Coverage/health                                 |
| model.inference.v1        | Optional high-volume sample for audit/drift     |

### 4.3 Ownership boundary

Data Engineering owns the hinge: **schemas, bus, stores, lineage, quality, retention, replay**. ML owns model semantics inside `detection.v1`. Risk owns assessment policy. Security owns who may read raw media. Systems Engineering owns ICD approval. Crossing those lines — e.g., putting swarm-threat rules in the normaliser — creates untestable coupling and prevents reuse of the same ingest path for simulation and live sensors.

### 4.4 ICD touchpoints for DE

- **ICD-01** Adapter → Normaliser: candidate JSON, at-least-once with idempotency, bounded local buffer.
- **ICD-02** Normaliser → Bus: `observation.v1`, quarantine on validation fail, registry subject compatibility BACKWARD.
- **ICD-03** Bus → processors: lag SLOs, idempotent handlers.
- **ICD-15** Sim fault injection: delay/drop/dup/corrupt — the DE proof crucible.

---

## 5. Canonical schemas and the `observation.v1` contract

---

### 5.1 Why canonicalisation is non-negotiable

Vendor-native schemas encode vendor worldviews: proprietary enums, nested binary blobs, sensor-local coordinates, and optional fields that become required only after a firmware update. If each downstream service adapts vendor payloads independently, the platform multiplies parsing bugs and diverges semantically (“confidence” meaning score vs probability vs SNR proxy).

A canonical observation is a **bounded context language** (in the DDD sense) for sensing facts. It is deliberately imperfect relative to any one sensor's richness; richness belongs in measurement and metadata with documented extension conventions, while the spine fields remain stable.

## 5.2 Normative field analysis (schemas/observation.v1.json)

| Field           | Role                            | Engineering notes                                                                   |     |     |     |           |            |        |      |
|-----------------|---------------------------------|-------------------------------------------------------------------------------------|-----|-----|-----|-----------|------------|--------|------|
| observation_id  | Global unique ID (UUID)         | Generated at normaliser (or adapter if guaranteed unique); never reused             |     |     |     |           |            |        |      |
| schema_version  | Contract version                | Const 1.0.0 in v1 schema; evolve via registry                                       |     |     |     |           |            |        |      |
| sensor_id       | Stable sensor identity          | Platform inventory key; not vendor serial alone                                     |     |     |     |           |            |        |      |
| source_type     | Modality enum                   | `radar`                                                                             | eo` | ir` | rf` | acoustic` | telemetry` | other` | sim` |
| timestamp_utc   | Event time (sensor phenomenon)  | RFC3339; after sync/conversion                                                      |     |     |     |           |            |        |      |
| received_at_utc | Processing-time receipt         | Platform clock; enables skew metrics                                                |     |     |     |           |            |        |      |
| idempotency_key | Dedupe key                      | Optional in schema but <b>operationally required</b> for at-least-once paths        |     |     |     |           |            |        |      |
| location        | CRS + lat/lon/alt + uncertainty | crs required; lat/lon nullable for non-geolocated RF/acoustic until multilateration |     |     |     |           |            |        |      |
| measurement     | Modality payload                | Open object; per-modality JSON Schemas as \$defs later                              |     |     |     |           |            |        |      |
| confidence      | [0,1] scalar                    | Document semantics per adapter; do not mix incompatible                             |     |     |     |           |            |        |      |

|                                         |                            |                                                           |
|-----------------------------------------|----------------------------|-----------------------------------------------------------|
|                                         |                            | meanings<br>silently                                      |
| quality                                 | Quality<br>flags/structure | Open object;<br>controlled<br>reason codes<br>recommended |
| provenance.adapter<br>/ adapter_version | Pedigree                   | Required                                                  |
| provenance.raw_ref                      | Pointer to raw<br>artefact | Object-store<br>URI when raw<br>retained                  |
| metadata                                | Non-<br>contractual<br>bag | Still subject to<br>size limits and<br>PII policy         |

additionalProperties: false on the root and on location/provenance is intentional Stage 0  
 strictness: unknown spine fields fail closed into quarantine rather than polluting the bus.

### 5.3 Open measurement vs closed spine — a deliberate trade-off

**Gain:** Adapters can ship modality detail without blocking the ICD spine.

**Loss:** Consumers may depend on undocumented measurement keys.

**Mitigation:** Per-source\_type extension schemas in /schemas/extensions/, versioned; contract tests for each adapter; fuzzing of size limits.

### 5.4 Identity and idempotency

Kleppmann emphasises that distributed systems naturally produce duplicates under retries. Counter-Swarm ICD-01 explicitly allows at-least-once delivery to the normaliser. Therefore:

- idempotency\_key should be deterministic from vendor event identity when available (e.g., hash of sensor\_id + vendor sequence + sensor timestamp).
- If the vendor lacks identity, the adapter must synthesise a key that is stable across retries of the *same* buffered item (store key with the buffer entry).
- The normaliser maintains a **dedupe cache** (Redis or compacted topic) keyed by idempotency\_key with TTL ≥ max retry horizon.
- observation\_id remains unique per accepted emit; duplicates of the key do not mint new IDs.

### 5.5 Null location is not a schema bug

RF and acoustic observations may lack immediate lat/lon. The schema allows null lat/lon while requiring crs. Fusion and tracking must tolerate geolocation that arrives later via association updates — as derived events, not by silently rewriting history without lineage. Prefer **append-only corrections** (observation.correction.v1 future) or track-level location over mutating prior observations in place.

## 5.6 Confidence semantics (epistemic hazard)

A single  $[0,1]$  field invites false commensurability. Stage 1 should add optional `confidence_semantics` in metadata or quality ("`snr_proxy`", "`classifier_proba`", "`operator_prior`"). Until then, adapters must publish a one-page semantics note; fusion should treat cross-modality confidence as **non-comparable** unless calibrated.

---

## 6. Heterogeneous failure: late, missing, duplicate, contradictory

---

### 6.1 Taxonomy

| Failure class | Example                                          | Primary control                                                                |
|---------------|--------------------------------------------------|--------------------------------------------------------------------------------|
| Late          | EO frame delayed 45s by backhaul                 | Watermarks; late side-channel; quality LATE                                    |
| Missing       | Radar plot gap during sector blanking            | <code>sensor.health.v1</code> ; risk missing-info; do not invent plots         |
| Duplicate     | Adapter retry after normaliser ACK loss          | Idempotency key dedupe                                                         |
| Contradictory | IR class “bird” vs RF class “uas”                | Preserve distributions; CLASS_CONFLICT; separate tracks if spatial assoc fails |
| Corrupt       | Truncated JSON, NaN altitudes                    | Quarantine with reason                                                         |
| Clock-skewed  | Sensor timestamp 2h off                          | Quality CLOCK_SKEW; possibly quarantine if beyond hard threshold               |
| CRS-wrong     | Lat/lon swapped or local metres labelled as 4326 | Validation; quarantine; never “fix” without audit                              |

### 6.2 Late data and watermarks

Following Akidau et al., define for each consumer:

- **Event time** = `timestamp_utc`
- **Processing time** = consumer local now (or `received_at_utc` for ingest skew)
- **Watermark** = claim that event time has advanced past  $T$  for a site/modality
- **Allowed lateness** = grace period for joining/updating

Counter-Swarm data-flow draft:

- Delayed obs **inside** watermark → late update; possible risk re-eval
- Delayed obs **beyond** watermark → side channel / reprocessing job; mark late

Do not pretend global watermarks across all sensors are equal to local ones. A site-level watermark that waits for the slowest sensor will stall EO-driven UI; a per-modality watermark with fusion-time reconciliation is more operable.

## 6.3 Missing data

Missingness is information. Suppressing gaps makes track coasting look like detection. Emit `sensor.health.v1` on expected heartbeat intervals; risk assessments already include “missing info” in the platform concept — DE must ensure health and coverage features are available as first-class inputs, not tribal knowledge.

## 6.4 Duplicates

Duplicates are not only transport artefacts; sensors themselves re-report. Dedupe layers:

1. Adapter buffer identity
2. Normaliser idempotency\_key
3. Consumer-side idempotency on `observation_id` for side effects

Metrics: `dedupe_drop_total` labelled by `sensor_id`.

## 6.5 Contradictions

From `architecture/data-flow.md`:

- Same track, conflicting class → keep distribution; flag `CLASS_CONFLICT`
- Spatially incompatible association → reject assoc; keep separate tracks

DE implication: the bus must carry **enough uncertainty and multi-hypothesis hooks** (even if v1 track schema starts simple) so that contradictions are not flattened into a single silent label. Flattening is a governance failure mode: operators cannot see disagreement.

## 6.6 Simulation as a first-class generator of these failures

ICD-15 requires fault injectors for delay/drop/dup/corrupt. A DE acceptance suite that only tests happy-path JSON is insufficient. Every quarantine reason code and watermark behaviour should have a sim scenario.

---

# 7. Event streaming: logs, partitions, delivery, and quarantine

---

## 7.1 Why a log-centric bus

Kreps' log abstraction gives Counter-Swarm:

- **Fan-out** without coupling producers to consumer sets
- **Replay** for incidents and regression
- **Temporal decoupling** (consumers at different latencies)
- **Operational inspection** (lag is visible)

Kafka-API compatibility preserves the option to run Apache Kafka or Redpanda; the trade-off document prefers Redpanda for lab/MVP ops simplicity while keeping the API (architecture/tradeoffs.md).

## 7.2 Topics (logical → physical)

| Logical event             | Suggested topic         | Notes                    |
|---------------------------|-------------------------|--------------------------|
| observation.v1            | obs.observation.v1      | Main spine               |
| observation.quarantine.v1 | obs.quarantine.v1       | Reason codes in payload  |
| detections/tracks/...     | det., <i>trk.</i> , ... | Separate ACL domains     |
| compact dedupe            | obs.idempotency.compact | Optional compacted topic |

Retention on observation topics: 7–30 days environment-specific (architecture/data-flow.md). This is **not** long-term archive; object store + audit cover longer horizons.

## 7.3 Producer settings (normaliser)

- Idempotent producer enabled (ICD-02)
- acks=all (or platform equivalent) for durability
- Bounded retries with metrics
- Payload size limits; large media by *raw\_ref*, not inline

## 7.4 Consumer discipline

- Checkpoint offsets **after** successful side effects (or use transactional patterns where available)
- Handlers idempotent on event IDs
- Lag SLO alerts (ICD-03); UI degraded banner when lag exceeds threshold

## 7.5 Quarantine path

Validation failure must not block the happy-path partition indefinitely. Quarantine messages should include:

- original payload (or safe truncated form)
- reason codes (machine-readable)
- adapter/provenance
- normaliser version
- hashes for integrity

Consumers of quarantine: DE quality dashboards, SG review sampling, adapter owners — **not** fusion.

## 7.6 Ordering guarantees (honest version)

Per-partition ordering only. Cross-sensor ordering is **not** provided by the bus. Fusion must use event time + association logic. Partitioning by `site_id` yields useful locality without implying a total site-wide order across producers unless all producers share a single partition key path (they do not).

## 7.7 Security notes for the bus

SASL/mTLS, per-principal ACLs (ICD-02/03). Raw EO clips in object store with stricter ACLs than track topics. Never log full payloads containing sensitive imagery at info level.

---

# 8. Geospatial hot state with PostgreSQL and PostGIS

---

## 8.1 Role of the hot store

The bus is optimised for write fan-out and replay; operators need:

- current tracks in a bounding box
- incident footprints
- geofence membership
- evidence link queries (`track_id` → `observation_ids`)
- config and inventory

PostgreSQL+PostGIS is the selected MVP hot store (ADR-002 direction): relational integrity for IAM-adjacent config and decisions, plus spatial indexes for map queries.

## 8.2 Suggested core tables (logical)

```
sensors(sensor_id PK, site_id, source_type, pose_geog, calibration_json, status)
observations_hot(observation_id PK, sensor_id, ts_event, geog, source_type,
confidence, quality_json, raw_ref, received_at)
  -- optional short retention partition; not full bus mirror
tracks(track_id PK, site_id, state_json, cov_json, geog, updated_at, quality_json)
track_evidence(track_id, observation_id, weight, primary key (track_id,
observation_id))
incidents(...)
alerts(...)
decisions(...) -- human.decision.v1 projections
quarantine_proj(...) -- optional
```

Use geography (WGS84) for global sites; geometry in a local projected CRS for precision work at a single site if survey requires it — but store the CRS explicitly and convert at boundaries.

### 8.3 Projection pattern

Consumers write projections **idempotently**. Tracks are state; observations on the bus are facts. Avoid treating PostGIS as the only place observations exist — that destroys replay purity. Hot `observations_hot` is a **cache for drill-down**, retained shorter than audit/object store.

### 8.4 Spatial integrity checks

- Reject lat outside  $[-90, 90]$ , lon outside  $[-180, 180]$  at normaliser
- Optional geofence: observation far from sensor feasibility envelope → quality flag or quarantine
- Altitude sanity relative to sensor capability

### 8.5 When Postgres becomes a liability

Under extreme insert rates for every observation, primary DB CPU/IO will contend with operator query latency. Mitigations in order:

1. Do not insert every observation into hot DB — sample or keep only evidence-linked
2. Partition tables by time
3. Add Timescale hypertables for observation history
4. Move analytical scans to export/OLAP

Premature introduction of a separate TSDB + geo DB splits lineage and multiplies backup stories — usually a bad early trade.

### 8.6 Transactions and decisions

Human decisions (ICD-10) should persist via transactional outbox: write decision + outbox row atomically; publisher emits `human.decision.v1` / handoff. This is the place for stronger consistency, not the sensor spine.

---

## 9. Time synchronisation, ordering, and watermarks

---

### 9.1 Two timestamps are mandatory

`timestamp_utc` (event time) and `received_at_utc` (receipt) enable:

- skew histograms per sensor
- detection of frozen clocks (event time stops while receipt advances)
- SLO measures on ingest delay

## 9.2 Sync architecture (practical)

| Layer             | Mechanism                            | Typical uncertainty       |
|-------------------|--------------------------------------|---------------------------|
| Sensor            | GNSS PPS / PTP aware where available | microseconds–milliseconds |
| Edge adapter host | chrony/NTP to site time source       | sub-ms to tens of ms      |
| Central services  | NTP                                  | tens of ms                |

Record estimated uncertainty in `quality.time_uncertainty_ms` when known. If unknown, flag `TIME_UNCERTAIN`.

## 9.3 Monotonicity checks

Per `sensor_id`, track last event time. Soft regression (small reordering) → accept with `OUT_OF_ORDER`. Hard jump backward beyond threshold → quarantine or accept with severe quality flag based on policy. Forward jumps may indicate clock reset — health event.

## 9.4 Fusion ordering rules (platform-level)

Document a single rule set, e.g.:

1. Association prefers event time within modality-specific gates.
2. UI ordering for “latest” uses track `updated_at` at processing time but displays event time.
3. Audit timelines show both.

Ambiguity here produces irreproducible after-action narratives.

## 9.5 Sim clocks

Simulation must be able to advance virtual event time independently of wall clock; adapters should stamp `timestamp_utc` from scenario time and `received_at_utc` from platform receipt — enabling deterministic replay tests.

---

# 10. Lineage, provenance, and evidence graphs

---

## 10.1 Minimum viable lineage

```
raw_ref → observation_id → detection_id → track_id → risk_id → alert_id
                                     ↘ recommendation_id → decision_id
```

Persist model versions on detections and risk method versions on assessments (architecture/data-flow.md lineage note). Adapter versions on observations are already required by schema.

## 10.2 Evidence arrays on tracks

`track.update.v1` should carry `evidence_obs_ids` (and optionally detection IDs). PostGIS `track_evidence` supports operator API drill-down: Operator API → hot links → obs IDs → bus/object fetch.

## 10.3 Raw artefact retention

`provenance.raw_ref` points to object storage. Without `raw_ref`, lineage stops at the canonical observation — often insufficient for EO disputes (“what did the camera actually see?”). Policy may hash or encrypt raw objects; DE still stores the pointer and content hash.

## 10.4 OpenLineage / internal lineage table

For Stage 1+, emit lineage events (job, inputs, outputs, `run_id`) for normalisation, projection, and export jobs. Even a single `lineage_edges` table beats tribal knowledge.

## 10.5 What not to do

- Do not rewrite observation rows to “fix” lineage after the fact without an append-only correction event.
- Do not let the AI assistant invent citations to observation IDs not returned by tools (assistant reads via allowlisted APIs only).

---

# 11. Data quality programmes and quarantine semantics

---

## 11.1 Checks at the normaliser (Stage 0 list)

From `docs/data/architecture.md`, expanded:

1. JSON Schema validity (`observation.v1`)
2. Required field completeness
3. Range checks (confidence, lat/lon, altitude bounds)
4. CRS presence and allowlist
5. Timestamp parseability; timezone must be UTC Z
6. Per-sensor monotonicity / skew thresholds
7. Rate anomaly detection (spike/drop vs baseline)
8. Payload size limits
9. `source_type` enum membership
10. Adapter allowlist (unknown adapter → quarantine)

## 11.2 Reason codes (recommended controlled vocabulary)

Examples: `SCHEMA_INVALID`, `RANGE_LAT`, `RANGE_LON`, `MISSING_CRIS`, `CLOCK_SKEW`, `RATE_ANOMALY`, `DUP_IDEMPOTENCY`, `PAYLOAD_TOO_LARGE`, `ADAPTER_UNKNOWN`, `NAN_MEASUREMENT`.

DUP\_IDEMPOTENCY may be metric-only drop rather than quarantine to avoid drowning the quarantine topic — but it must be counted.

### 11.3 Quality vs quarantine

- **Quarantine:** unsafe to enter fusion happy path
- **Quality flags on valid obs:** usable with degraded trust (CLOCK\_SKEW mild, LATE, OUT\_OF\_ORDER)

Risk and UI should surface quality aggregates; hiding flags creates false certainty.

### 11.4 SLOs and metrics

- Quarantine rate per sensor / adapter
- Schema fail rate
- Ingest delay p50/p95 (received\_at - timestamp)
- Dedupe rate
- Bus lag per consumer group
- Hot DB projection error rate

These belong on the observability backbone (ICD-14 / OTLP).

---

## 12. Retention, tiering, and cost governance

---

### 12.1 Draft retention (from platform docs)

| Store              | Retention draft                                            |
|--------------------|------------------------------------------------------------|
| Event bus          | 7–30 days                                                  |
| Hot DB operational | ~90 days online; archive after                             |
| Audit              | Long-term ( $\geq 1$ year default design; customer policy) |
| Raw object store   | Tiered; cost-governed                                      |
| Model artefacts    | Registry policy                                            |

### 12.2 Tiering strategy

- **Hot:** PostGIS operational state + short obs cache
- **Warm:** Recent Parquet/object partitions of observations and tracks for replay beyond bus retention
- **Cold:** Glacier-like classes for compliance raw media
- **Analytic:** Export subsets for DS — not authoritative for audit disputes

### 12.3 Cost governors

- Max raw EO retention minutes at full rate unless incident-pinned

- Incident pin API: extend retention for involved `raw_refs`
- Quotas per site

## 12.4 Legal/hold

SG defines hold policy; DE implements **pin flags** that block lifecycle deletion. Silent deletion of pinned evidence is a critical failure.

---

## 13. Replay, reprocessing, and after-action reconstruction

---

### 13.1 Replay use cases

1. Incident after-action review
2. Regression of tracking/fusion after algorithm change
3. Training of operators on recorded scenarios
4. DE proof of quarantine/watermark behaviour

### 13.2 Replay modes

| Mode                     | Mechanism                                           | Fidelity                                |
|--------------------------|-----------------------------------------------------|-----------------------------------------|
| Bus time-range replay    | Consumer resets offsets / uses tools to read window | High for events still in retention      |
| Object-store warm replay | Re-publish from Parquet/canonical archive           | High if archive complete                |
| Snapshot + bus           | Restore PostGIS snapshot then play forward          | Needed when state bootstrapping matters |
| Sim deterministic replay | Re-run scenario RNG seed                            | Best for CI                             |

### 13.3 Determinism limits

Floating-point association and parallel consumers mean **bit-identical** track IDs may not reproduce. Require **semantic reproducibility**: same evidence sets within tolerance, same alert tier under frozen policy versions. Record versions of fusion/risk configs in the replay bundle.

### 13.4 Replay bundles

An exportable bundle for an incident:

- event segments (canonical JSON)
- config/policy versions
- model versions
- PostGIS snapshot or track evidence tables
- hashes and manifest

Analysts and SG consume bundles; they should not require production DB credentials.

---

## 14. Schema evolution and registry policy

---

### 14.1 Compatibility mode

ICD-02: subject `observation-value` with compatibility **BACKWARD** within major. Practical rules:

- Add optional fields safely
- Do not rename/remove required fields without major bump
- Consumers ignore unknown fields (ICD conventions)
- Producers during rollout may emit N while consumers still understand N-1

### 14.2 Expand/contract for databases

Hot DB migrations follow expand/contract:

1. Expand: add nullable columns / new tables
2. Dual-write or backfill
3. Switch readers
4. Contract: remove old columns after consumers gone

Never combine destructive contract with producer release in one step.

### 14.3 `schema_version` field vs registry

The const `1.0.0` in JSON Schema is the **logical** version. Registry subject versions are artefact versions. Keep a mapping table in docs when they diverge (e.g., registry v4 still logical 1.1.0).

### 14.4 Breaking changes playbook

1. Draft `observation.v2` alongside v1
2. Dual-publish period or dual-topic
3. Reprocess warm archive if semantic reinterpretation required
4. Quarantine v1-only adapters after deadline

### 14.5 Contract tests as the real enforcement

Section 17 makes tests the gate. A registry without CI is documentation cosplay.

---

## 15. Architecture alternatives and justified recommendation

---

### 15.1 Approach A — Database-centric ingest

Sensors write directly to PostGIS; fusion polls tables.

- **Components:** adapters, DB, workers
- **Gains:** simple mental model; ACID for single-node
- **Losses:** poor fan-out; painful replay; rate coupling to DB; hard quarantine stream
- **Scaling:** vertical then shatter
- **Liability:** becomes monolith bottleneck
- **Verdict:** Reject for production path (acceptable only for tiny spikes)

### 15.2 Approach B — Fine-grained streaming + specialised stores everywhere

Kafka + ClickHouse + RedisGEO + object store + feature store on day one.

- **Gains:** each system optimised
- **Losses:** operational burden; lineage fragmentation; consistency puzzles
- **Verdict:** Reject for MVP; revisit individual pieces under measured pressure

### 15.3 Approach C — Log-centric bus + PostGIS projections + object cold store (selected)

- **Components:** adapters, normaliser, Kafka-API bus, Postgres+PostGIS, S3 API, optional Timescale later
- **Gains:** matches sensor asynchrony; replay; clear ICD boundaries; team-operable
- **Losses:** eventual consistency between bus and projections; requires consumer discipline
- **Failure modes:** lag, projection bugs, retention gaps — all mitigable with metrics and bundles
- **Verdict: Recommended**, aligned with ADR-001/002/003

### 15.4 Explicit trade-offs (selected design)

| Decision                    | Gains              | Losses             | Later pain if wrong                         |
|-----------------------------|--------------------|--------------------|---------------------------------------------|
| Kafka-API bus               | Replay, ecosystem  | Ops weight         | Migration to non-log bus loses replay story |
| PostGIS hot                 | One system geo+rel | TS write ceilings  | Add Timescale/export under pressure         |
| Strict spine schema         | Safety             | Adapter friction   | Loosen measurement only, not spine          |
| At-least-once + idempotency | Simple producers   | Dedupe complexity  | Exactly-once myths without keys fail harder |
| Quarantine topic            | Visibility         | Noise if mis-tuned | Silent drop destroys trust                  |

---

## 16. Failure modes and operational burden

---

### 16.1 Ranked failure modes

| Rank | Failure                                                      | Detection                              | Mitigation                                    |
|------|--------------------------------------------------------------|----------------------------------------|-----------------------------------------------|
| 1    | Silent data loss (drop without metric)                       | Produce/consume audit counters diverge | Quarantine-or-count rule; end-to-end canaries |
| 2    | Clock poison (bad timestamps enter fusion)                   | Skew monitors; sim tests               | Hard quarantine thresholds                    |
| 3    | Projection divergence (DB $\neq$ bus truth)                  | Replay diff jobs                       | Rebuild projections from bus/warm             |
| 4    | Retention gap (incident older than bus TTL, no warm archive) | Bundle export failures                 | Warm archive job SLO                          |
| 5    | Schema break in producer                                     | Registry CI; consumer error spikes     | BACKWARD gate; kill switch                    |
| 6    | Quarantine flood masking real issues                         | Rate alerts                            | Adaptive sampling; adapter disable            |
| 7    | Hot DB overload from obs inserts                             | DB metrics                             | Stop mirroring all obs                        |
| 8    | ACL misconfig exposing raw media                             | SG audits                              | Separate buckets/topics                       |

### 16.2 Partial outage behaviour

- Bus down: adapters buffer bounded; shed oldest with metric (ICD-01); never crash into undefined state
- PostGIS down: continue bus ingest; UI read-only/degraded; block decisions if approval store down (ICD-09/10)
- Object store down: accept obs without raw\_ref only if policy allows; else quarantine RAW\_STORE\_UNAVAILABLE

### 16.3 Operational burden acceptance

Owning Kafka-API + Postgres + object store is already non-trivial. Adding multi-region active-active in MVP contradicts architecture non-goals. Prefer documented runbooks for single-region restore and replay.

---

## 17. Proof: contract tests, property tests, and evaluation harnesses

---

### 17.1 Contract tests (merge gates)

For every producing service, before schema change merges (architecture/interfaces.md §5):

1. **Schema validate** golden fixtures for observation.v1
2. **Negative fixtures** for each quarantine reason

3. **Consumer compatibility:** old consumer library can read new optional fields
4. **Idempotency:** duplicate key → single downstream effect
5. **ICD latency smoke** where applicable (not full NFR soak in unit CI)

## 17.2 Property-based tests

- For any valid observation, round-trip JSON encode/decode preserves spine fields
- Lat/lon swapping detection on feasibility envelopes
- Monotonicity detector classification across randomised sequences

## 17.3 Sim fault suite (proof of resilience)

| Scenario          | Expected                                      |
|-------------------|-----------------------------------------------|
| Dup 10%           | Dedupe metric; no double tracks from same key |
| Delay 60s         | Late handling path; quality LATE              |
| Corrupt 1%        | Quarantine SCHEMA_INVALID/corrupt             |
| Clock skew +10min | CLOCK_SKEW flag or quarantine per policy      |
| Drop 20%          | Health/missing-info reflects gaps             |

## 17.4 Replay differential proof

Freeze a scenario bundle; run fusion twice; compare evidence sets and alert tiers under version-pinned configs. Fail CI on unexplained drift.

## 17.5 What “proof” does not mean

These tests do not prove correct tracking physics. They prove **platform honesty**: contracts, quarantine, dedupe, and replay mechanics — the substrate ML/DS evaluation stands on.

---

# 18. Build guide (incremental delivery)

---

## 18.1 Stage 0 — Contracts and honesty

1. Freeze `observation.v1` JSON Schema (done as draft in repo).
2. Document quarantine reason codes.
3. Stand up Kafka-API bus in lab (Redpanda acceptable).
4. Implement normaliser validate+quarantine+produce.
5. One sim adapter + one “vendor-shaped” fixture adapter.
6. Contract tests in CI.
7. Metrics: quarantine rate, produce errors, skew histogram.

**Exit criteria:** 24h sim soak with fault injection; no silent drops.

## 18.2 Stage 1 — Projections and operator drill-down

1. PostGIS schema for tracks, evidence, sensors.
2. Track projector consumer (even if tracker is classical/simple).
3. Operator API: query track → evidence → observation fetch.
4. Object store raw\_ref for EO fixtures.
5. Warm archive job: bus → Parquet daily.

**Exit criteria:** After-action drill reconstructs a scripted incident without DB archaeology by hand.

## 18.3 Stage 2 — Quality programme and lineage UI/API

1. Rate anomaly detectors.
2. Lineage API decision → observations.
3. Quarantine console for DE/SG.
4. Expand measurement extension schemas per modality.

## 18.4 Stage 3 — Hardening

1. Retention governors and incident pinning.
2. Expand/contract migration drills.
3. Timescale decision under measured load (only if needed).
4. Multi-site partition strategy review.

## 18.5 Explicit non-builds (yet)

- Unbounded raw topics without retention
- PII/imagery in clear debug logs
- Risk rules in normaliser
- Custom database engine
- Multi-region active-active

## 18.6 Suggested repository layout (engineering)

```
schemas/  
  observation.v1.json  
  quarantine.v1.json      # to add  
  extensions/radar.v1.json # to add  
docs/data/  
  architecture.md  
  reason-codes.md        # to add  
  retention.md            # to add  
services/normaliser/     # future implementation root  
tests/contracts/  
  observation/  
infrastructure/           # bus, postgres, minio compositions
```

---

## 19. Pre-ship gate and open decisions

---

| Question                  | Answer                    | Justification                                              |
|---------------------------|---------------------------|------------------------------------------------------------|
| Failure modes understood? | YES (draft)               | Section 16; needs live soak to confirm rates               |
| Observable in prod?       | YES if OTLP metrics land  | Quarantine/lag/skew required before production sensors     |
| Safe rollback?            | YES in design             | Schema dual-publish; projection rebuild; model/config pins |
| Complexity proportional?  | YES for selected approach | Rejected multi-store sprawl                                |
| Want 3-year ownership?    | YES if spine stays stable | Canonical observation + log is the hinge                   |

### Open decisions to close with Systems Engineering:

1. Avro vs JSON on the wire for ICD-02
  2. Partition key finalisation (`site_id` vs `sensor_id`)
  3. Hard vs soft thresholds for `CLOCK_SKEW` quarantine
  4. Warm archive format (Parquet schema) ownership with DS
- 

## 20. Conclusion

---

Heterogeneous multi-sensor fusion platforms do not earn trust through dashboards alone. They earn trust when every observation is a **versioned fact**, when failures become **quarantine and metrics** rather than silence, when geospatial state is a **projection** of a replayable log, and when a human decision can be unfolded into evidence without mythology.

Counter-Swarm Defence already points in the right direction: `observation.v1` as hinge, Kafka-API bus, quarantine event family, PostGIS hot store, object-store raw references, and explicit refusal to bury risk policy in ingest. This monograph has argued — with established systems literature and platform-mapped engineering detail — how to make that direction operable under late, missing, duplicate, and contradictory data; how to evolve schemas; how to prove contracts; and how to build in stages without drowning the team.

The work that remains is not conceptual novelty. It is disciplined implementation: reason codes, CI contract gates, warm archive SLOs, and fault-injected soaks until silent loss is structurally difficult. That is the data-engineering contribution to defensive decision-support that survives audit, ownership, and contact with real sensors.

---

## 21. References

---

- Akida, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernández-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Nordstrom, P., & Whittle, S. (2015). The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12), 1792–1803.
- Bar-Shalom, Y., Willett, P. K., & Tian, X. (2011). *Tracking and Data Fusion: A Handbook of Algorithms*. YBS Publishing.
- Blackman, S., & Popoli, R. (1999). *Design and Analysis of Modern Tracking Systems*. Artech House.
- Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). Apache Flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4), 28–38.
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, D., Nagle, D., Quinlan, S., ... Woodford, D. (2012). Spanner: Google’s globally distributed database. *ACM Transactions on Computer Systems*, 31(3), Article 8. (Conference version: OSDI 2012.)
- Hall, D. L., & Llinas, J. (1997). An introduction to multisensor data fusion. *Proceedings of the IEEE*, 85(1), 6–23.
- IEEE Std 1588. (various editions). *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. IEEE.
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O’Reilly Media.
- Kreps, J. (2013). *The Log: What every software engineer should know about real-time data’s unifying abstraction*. Engineering blog essay, Confluent/LinkedIn lineage.
- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. *Proceedings of the NetDB Workshop*.
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.
- Liggins, M. E., Hall, D. L., & Llinas, J. (Eds.). (2008). *Handbook of Multisensor Data Fusion: Theory and Practice* (2nd ed.). CRC Press.
- Marz, N., & Warren, J. (2015). *Big Data: Principles and best practices of scalable realtime data systems*. Manning.
- Mills, D. L. (1991). Internet time synchronization: The Network Time Protocol. *IEEE Transactions on Communications*, 39(10), 1482–1493.

Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka: The Definitive Guide*. O'Reilly Media. (2nd ed. later with additional authors; 1st ed. cited as landmark.)

OGC. (2010). *OpenGIS Implementation Standard for Geographic information – Simple feature access – Part 1: Common architecture* (Version 1.2.1). Open Geospatial Consortium.

PostGIS Project Steering Committee. (ongoing). *PostGIS documentation*.  
<https://postgis.net/documentation/>

RFC 3339. Klyne, G., & Newman, C. (2002). *Date and Time on the Internet: Timestamps*. IETF.

Steinberg, A. N., Bowman, C. L., & White, F. E. (1999). Revisions to the JDL data fusion model. *Proceedings of SPIE*, 3719, 430–441.

Waltz, E., & Llinas, J. (1990). *Multisensor Data Fusion*. Artech House.

---

## Appendix A — Mapping matrix: monograph controls → repo artefacts

| Control                         | Artefact                                                 |
|---------------------------------|----------------------------------------------------------|
| Canonical spine                 | schemas/observation.v1.json                              |
| Event families / contradictions | architecture/data-flow.md                                |
| ICD produce/consume             | architecture/interfaces.md                               |
| Store tiers                     | docs/data/architecture.md, architecture/tradeoffs.md     |
| DE ownership                    | docs/systems/agent-responsibility-matrix.md              |
| Quarantine                      | observation.quarantine.v1 (logical; schema file Stage 1) |

---

## Appendix B — Example quarantine payload (informative)

```
{
  "quarantine_id": "uuid",
  "schema_version": "1.0.0",
  "reason_codes": ["CLOCK_SKEW", "RANGE_LAT"],
  "observed_at_utc": "2026-09-19T20:00:00Z",
  "adapter": "adapter-radar-acme",
  "adapter_version": "0.3.1",
  "sensor_id": "siteA-radar-01",
  "payload_hash": "sha256:...",
  "payload_truncated": {},
  "normaliser_version": "0.1.0"
}
```

## Appendix C — Normaliser pseudocode (informative)

---

```
function handle_candidate(c):
    if not schema_valid(c): return quarantine(c, SCHEMA_INVALID)
    if duplicate(c.idempotency_key): metric(dedupe); return
    q = []
    q += check_ranges(c)
    q += check_crs(c)
    q += check_clock(c)
    q += check_rate(c)
    if hard_fail(q): return quarantine(c, q)
    obs = enrich(c) # observation_id, received_at_utc, quality flags
    produce(topic_observation, obs)
    if soft_flags(q): metric(quality_flags)
```

---

## Appendix D — Extended discussion: encoding, compression, and payload budgets

---

### D.1 Encoding choices on the bus

Stage 0's normative file is JSON Schema validating JSON objects. That choice optimises **human debuggability** and contract-test clarity. Kleppmann's encoding chapter distinguishes language-specific serialisation (fragile), textual JSON/XML (ubiquitous, verbose), and binary schemata (Avro/Protobuf/Thrift) with sharper evolution tooling.

For Counter-Swarm:

- **Keep logical fields identical** across encodings.
- Prefer JSON for lab and early integration.
- Introduce binary encoding when profiling shows CPU/network saturation on observation topics — typically after EO metadata growth or very high-rate radar plots.
- Never allow two conflicting logical schemas for the same `schema_version`.

Compression (zstd/lz4 at the bus layer) is an operations dial, not a schema decision. Measure consumer CPU before enabling aggressive compression on small messages (radar plots may be tiny; compression overhead can dominate).

### D.2 Payload budgets

Inline measurement must have hard ceilings (e.g., 64 KiB) enforced at the normaliser. Imagery, spectrograms, and long IQ captures belong in object storage with `raw_ref`. Violations quarantine as `PAYLOAD_T00_LARGE`. This single rule prevents a class of outages where one EO adapter melts the bus.

## D.3 Header vs body metadata

Kafka-API headers can carry `schema_version`, `site_id`, `sensor_id`, and tracing IDs without re-parsing bodies for routing. Body remains the authoritative business payload. Document which fields are duplicated into headers; divergence is a defect.

---

## Appendix E — Extended discussion: multi-site and edge buffering

---

### E.1 Central-primary hybrid

Architecture recommends central-primary MVP with edge adapters (`architecture/system.md`). Data engineering consequences:

- Edge buffers are **bounded** queues with shed-oldest policy and metrics.
- Store-and-forward must preserve idempotency keys across reconnect.
- Do not run divergent normaliser versions at edge and centre without dual-publish rules.

### E.2 Site as tenancy unit

`site_id` should appear on observations (via metadata or future required field) for ACL and partition locality. Multi-site fusion across posts is a central function; edge fusion, if any, must emit events reconcilable centrally (no silent ID spaces).

### E.3 Denied, disrupted, intermittent, limited (DDIL) networks

Under backhaul impairment:

- Prefer local health and track coasting indicators over inventing central certainty.
  - Queue observations; when flush resumes, expect burst duplicates and late storms — watermark and dedupe configurations must be sized for flush, not only steady state.
  - Warm archive at edge is optional Stage 3+; do not invent a second long-term system of record casually.
- 

## Appendix F — Extended discussion: analytics export without dual truth

---

DS and ML need offline datasets. The correct pattern:

1. Operational truth remains bus + audit + pinned raw objects.
2. Periodic export writes Parquet (observations, tracks, labels) to an analytics bucket.
3. Notebooks read exports; they do not write back into operational topics except through governed reprocessing jobs.

Kimball-style conformed dimensions (sensor, site, modality) help evaluation joins, but the warehouse is not the court of appeal for operator disputes — audit bundles are.

Lambda's batch layer is tempting for "recompute everything." Kappa-style replay from warm canonical archives usually fits Counter-Swarm better: one modelling of events, multiple consumers, including batch exporters.

---

## Appendix G — Worked scenarios

---

### G.1 Duplicate radar plots after link flap

1. Adapter buffers plots while uplink down.
2. On reconnect, retries overlapping sequence.
3. Normaliser dedupes via idempotency\_key; metric increments.
4. Fusion never sees doubles; track continuity preserved.

**Failure if broken:** split tracks, inflated counts, false swarm density.

### G.2 Contradictory EO vs RF classification

1. Both observations valid schema-wise.
2. Association links both to track T.
3. Track carries class distribution; CLASS\_CONFLICT quality.
4. Risk may raise missing-info / uncertainty — not a forged single label.

**Failure if broken:** UI shows "UAS" with no dissent; audit cannot explain error.

### G.3 Late IR frame after watermark

1. Fusion closed a window; risk emitted.
2. Late IR arrives with LATE flag.
3. Reprocessing job updates track evidence; may emit revised risk with linkage to prior assessment IDs.
4. Operators see revision marker — not silent history rewrite.

### G.4 Schema-invalid acoustic burst

1. Candidate fails JSON Schema.
  2. Quarantine event with SCHEMA\_INVALID.
  3. Adapter owner alerted via quarantine rate SLO.
  4. Fusion unaffected.
-

## Appendix H — Security and privacy intersections for DE

---

Without repeating the full threat model (SG-owned), DE must implement:

- Separate object-store prefixes/buckets for raw EO with stricter IAM
- Redaction rules for logs (no full frames)
- Encryption at rest for cold raw
- Retention deletes that honour legal hold pins
- Bus ACLs so that recommendation services cannot read raw imagery topics if separated

Data quality tooling must not become a backdoor data exfiltration UI; quarantine consoles need RBAC.

---

## Appendix I — Glossary

---

| Term                  | Meaning                                               |
|-----------------------|-------------------------------------------------------|
| Event time            | Sensor phenomenon time ( <code>timestamp_utc</code> ) |
| Processing time       | When a component handles a record                     |
| Watermark             | Progress belief about event time                      |
| Canonical observation | Platform-standard sensing fact                        |
| Quarantine            | Side path for invalid/suspicious candidates           |
| Projection            | Derived store updated from events                     |
| Idempotency key       | Stable key for dedupe across retries                  |
| Pedigree / lineage    | Trace from outputs to inputs and versions             |
| Warm archive          | Mid-tier retained canonical events beyond bus TTL     |

---

## Appendix J — Evaluation checklist for external reviewers

---

1. Are citations real landmark works?
  2. Does the design map to `observation.v1` fields without inventing conflicting spines?
  3. Is quarantine mandatory on hard validation failure?
  4. Are late/duplicate/contradiction behaviours testable in sim?
  5. Is risk policy excluded from normaliser?
  6. Is replay concrete (bundle contents) rather than aspirational?
  7. Are trade-offs explicit about PostGIS write ceilings?
  8. Would you accept 3-year ownership of this design?
-

## Appendix K — Deep dive: canonical schema governance as an industrial process

---

### K.1 Schemas as interfaces, not documents

In software engineering, an interface that is not enforced is a rumour. Counter-Swarm’s `observation.v1` JSON Schema must be treated as a **machine-checked boundary**. The governance loop is:

1. Proposal: change request with compatibility analysis (BACKWARD / FORWARD / FULL / BREAKING).
2. Review: Data Engineering + at least one producing adapter owner + one consuming service owner (tracking or detection).
3. Fixture update: golden and negative fixtures land in the same pull request as the schema diff.
4. Registry publish: subject version increments; logical `schema_version` updates only when semantics warrant.
5. Rollout: producers and consumers deploy in an order consistent with compatibility mode.
6. Sunset: removal of dual-write paths after metrics show zero N-1 traffic.

Kleppmann’s guidance on schema evolution is often summarised as “prefer additive changes.” That slogan is necessary but insufficient for multi-sensor platforms because **additive fields can still be semantically breaking**. Adding `confidence_calibrated` while leaving old `confidence` ambiguous can cause fusion to prefer the wrong field. Therefore every additive field requires a short semantics note and a consumer adoption test — not merely registry greenness.

### K.2 Owned vocabularies versus open maps

The spine uses closed enums for `source_type` and closed objects for `location` and `provenance`. The open maps `measurement`, `quality`, and `metadata` are pressure valves. Without discipline, pressure valves become sewage:

- Adapters invent parallel keys (`lat` inside `measurement` while spine `location` is null).
- Confidence clones proliferate (`score`, `p`, `conf`).
- Quality flags become free text, destroying dashboards.

Mitigation pattern:

- Publish per-modality extension schemas under `schemas/extensions/`.
- CI: if `source_type==radar`, validate `measurement` against `radar.v1` when present.
- Linting job weekly: scan bus sample for undeclared keys above frequency threshold; open tickets automatically.

### K.3 Version coupling across event families

`observation.v1` will not be the only contract. `detection.v1` and `track.update.v1` will reference observation IDs and sometimes embed denormalised snippets. Rules:

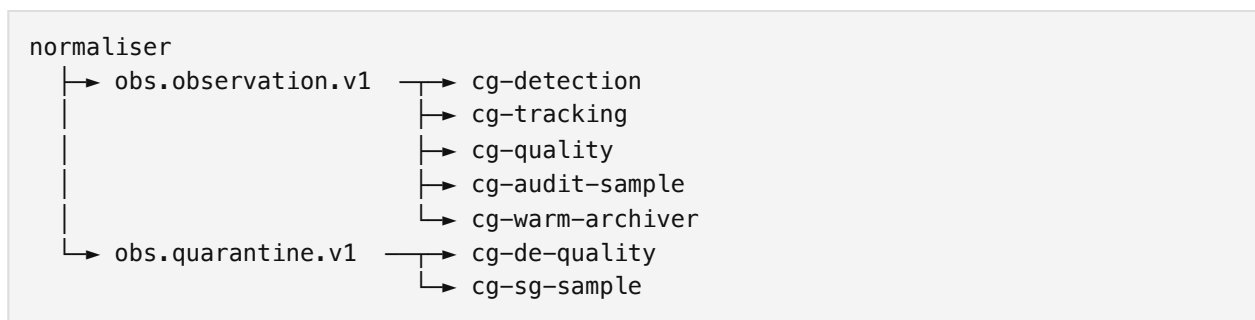
- Embedded snippets are **caches**, not new sources of truth.
- If observation semantics change in a major version, derived families need an explicit migration note even if their own JSON Schema is untouched.
- Evidence links should prefer IDs over copied payloads to reduce migration blast radius.

## K.4 Human-readable ICD versus executable schema

architecture/interfaces.md describes latency, auth, and failure behaviour that JSON Schema cannot express. Keep both: schema for structure; ICD for protocol. Contract tests should cover a slice of ICD behaviour (idempotency, quarantine on invalid) while soak tests cover latency SLOs.

## Appendix L — Deep dive: streaming topology, consumer groups, and backpressure

### L.1 Topology sketch for Stage 1



Separate consumer groups are mandatory so detection lag does not block warm archiving. A common failure mode in early Kafka adopters is a single “god consumer” that does projection, detection, and archive in one process — transforming a distributed log back into a monolith with worse debuggability.

### L.2 Backpressure and load shedding

When consumers lag:

1. Alert on lag age and lag messages.
2. UI degraded banner (ICD-03) — operators must know freshness is impaired.
3. Optional: detection may shed non-critical modalities under declared policy; tracking should not invent measurements to “keep up.”
4. Never delete bus data to catch up without warm archive confirmation.

Adapters already shed-oldest on local buffers (ICD-01). The platform must surface shed metrics beside quarantine metrics; otherwise operators optimise the wrong knob.

### L.3 Exactly-once, idempotent producers, and transactional outboxes

Kafka's idempotent producer prevents duplicate writes from **producer retries** to the same partition under the producer session — it does not dedupe **application-level** duplicates from adapter retries with new producer records. That is why `idempotency_key` remains mandatory operationally.

For approval/integration, prefer transactional outbox in Postgres: the decision row and outbox row commit together; a publisher relays to the bus. This matches Kleppmann's dual-write warning: writing DB and bus independently without a linking pattern creates divergence.

### L.4 Compacted topics for reference data

Sensor inventory, geofences, and calibration blobs change slowly. Compacted topics (or PostGIS as primary with bus notifications) work well. Do not compact observation topics — that destroys history required for replay.

### L.5 Multi-cluster and mirror risk

MVP is single-region. If a customer later demands DR, mirror warm archives and configure offset translation carefully. Naïve mirroring without schema registry replication breaks consumers. Treat multi-cluster as a programme, not a checkbox.

---

## Appendix M — Deep dive: PostGIS physical design and query paths

---

### M.1 Geography versus geometry

geography types on WGS84 simplify global deployments and correct distance semantics for geofence checks across long spans. geometry in a local projected CRS (e.g., UTM zone) can yield faster and more precise local analytics for a single site survey grid. Recommendation:

- Store operational positions as geography(PointZ, 4326) or 2D if altitude separate.
- If a site requires survey-grade local ops, maintain a **derived** geometry column in local CRS updated by trigger or application — do not invent a second write path from adapters.

### M.2 Indexing strategy

- GiST on track geography for viewport queries.
- BRIN on time columns for append-heavy observation caches.
- B-tree on sensor\_id, track\_id, incident\_id.
- Partial indexes for status = active tracks to keep operator maps fast.

## M.3 Partitioning

Time-partition observations\_hot monthly; drop partitions beyond hot retention rather than row-by-row delete. Tracks may remain unpartitioned longer if cardinalities are modest; partition by site if multi-tenant tables become large.

## M.4 Avoiding the “bus mirror” anti-pattern

Inserting every observation into PostGIS recreates the bus inside a database that also serves transactional approval traffic. Prefer:

- Insert observations only when linked as evidence, or
- Insert with aggressive TTL partitions, or
- Serve observation drill-down from warm Parquet/object index for older IDs and keep only recent in DB.

Operator API can abstract the tier: “get observation by ID” checks hot then warm.

## M.5 Spatial validation beyond schema

Schema cannot encode “this radar cannot physically observe that lat/lon given pose and Earth curvature constraints” without context. Feasibility filters should live in the normaliser or a quality service with access to sensor pose. Failures become FEASIBILITY\_VIOLATION quality or quarantine depending on severity. This is a high-value anti-spoof / anti-misconfig control even in defensive civilian deployments.

## M.6 Backup and restore

PostGIS backups must be tested with a restore drill that includes spatial indexes. Replay bundles should not assume production DB availability; export evidence tables into the bundle format.

---

# Appendix N — Deep dive: time, synchronisation engineering, and testability

---

## N.1 Threats to time honesty

1. GNSS jamming/spoofing at the sensor (sensor time jumps).
2. NTP step corrections on hosts after long isolation.
3. Virtualisation clock drift on overloaded VMs.
4. Sim/real mix-ups (scenario time written into live topics).
5. Human misconfiguration of timezone (emitting local time as if UTC).

Controls: quality flags; hard quarantine beyond threshold; inventory field `time_sync_mode` (gnss, ptp, ntp, none); refuse `source_type!=sim` events from sim-only credentials.

## N.2 Representing uncertainty

Spanner's TrueTime teaches a design moral even when you are not building Spanner: **intervals beat lies**. Where possible, adapters should publish:

```
quality.time_uncertainty_ms = 5
quality.time_sync_mode = "ptp"
```

Fusion gates should widen association windows when uncertainty is large rather than assuming fictitious precision.

## N.3 Monotonicity state store

Normaliser monotonicity checks need state: last event time per sensor. Options:

- Redis with TTL and persistence caution
- Compacted Kafka topic
- In-memory with peer broadcast (weak)

Redis is acceptable for MVP if loss of state only causes temporary soft-flag noise after restart — document that behaviour.

## N.4 Ordering for audit narratives

After-action reports should sort by event time within a track's evidence, while showing ingest delay as a separate column. Sorting only by processing time misleads commanders about the physical sequence.

---

## Appendix O — Deep dive: lineage graphs, model versions, and decision defensibility

---

### O.1 Why lineage is a safety-adjacent property

In defensive decision-support, a recommendation without evidence is a liability. SG will ask: which observations, which model version, which policy ID? If DE cannot answer via API, humans will screenshot maps — an unsearchable “lineage medium.”

### O.2 Graph shape

Model as a directed acyclic graph (DAG) of artefacts:

- Nodes: raw objects, observations, detections, tracks, indicators, risks, alerts, recommendations, decisions
- Edges: DERIVED\_FROM, EVIDENCE\_FOR, USES\_MODEL, USES\_POLICY
- Attributes: versions, timestamps, hashes

Store edges in Postgres for queryability; optionally emit OpenLineage for data jobs (archives, training exports).

### O.3 Model and policy pins in replay

Replaying tracks with a newer tracker without recording the version proves nothing about past operator experience. Replay bundles must pin:

- tracker algorithm version
- fusion association gates
- risk policy ID
- alert policy ID

### O.4 Incomplete lineage handling

If `raw_ref` is missing, lineage API must return an explicit `raw_unavailable` rather than omitting the field. Absence should be loud.

---

## Appendix P — Deep dive: quality scoring programmes and operator-visible degradation

---

### P.1 From binary quarantine to graded trust

Not all defects are equal. A mild `OUT_OF_ORDER` of 200 ms on radar is routine; a 15-minute clock skew is poison. Publish a mapping:

| Code              | Default action       | Operator visibility |
|-------------------|----------------------|---------------------|
| SCHEMA_INVALID    | Quarantine           | DE console          |
| CLOCK_SKEW hard   | Quarantine           | DE + site health    |
| CLOCK_SKEW soft   | Accept + flag        | Track quality chip  |
| LATE inside grace | Accept + flag        | Optional            |
| LATE beyond grace | Side channel         | Analyst tooling     |
| CLASS_CONFLICT    | Accept               | Explicit UI dissent |
| RATE_ANOMALY      | Accept or quarantine | Health banner       |

Product Design owns chips/banners; DE owns the codes and metrics truthfulness.

### P.2 Canary observations

Inject synthetic canary observations with known IDs through a privileged path at low rate. If they fail to appear in warm archive or projections within SLO, page DE. Canaries detect silent loss better than dashboards alone.

## P.3 Quarantine review workflow

Quarantine without review becomes a write-only graveyard. Weekly triage:

- Top reason codes by volume
- Top sensors
- Whether schema bugs or real sensor faults dominate

Feed results into adapter backlogs.

---

## Appendix Q — Deep dive: retention mathematics and incident pinning

---

### Q.1 Sizing sketch (illustrative)

Suppose a site emits 200 observations/s average. At ~1 KB canonical JSON average:

- ~200 KB/s  $\approx$  17 GB/day on the bus before replication factor.
- With RF=3, raw cluster disk ~50 GB/day order-of-magnitude before indexes/overhead.
- 14-day retention  $\approx$  0.7 TB scale — lab-feasible; multi-site multiplies.

EO raw frames are the real cost driver. Separate quotas:

- Canonical events: time retention
- Raw media: minutes at full rate + pin on incident

### Q.2 Pin API semantics

```
POST /api/v1/incidents/{id}/pin-evidence
→ marks raw_ref and event segments with delete_earliest_at = null / far future
```

Unpin requires SG role. Lifecycle workers must be idempotent and skip pinned objects.

### Q.3 GDPR-like and privacy constraints

Even defensive systems may capture bystanders in EO. Retention minimisation is not only cost — it is governance. DE implements technical controls; SG sets policy.

---

## Appendix R — Deep dive: replay engineering and differential evaluation

---

### R.1 Replay driver

A replay driver should:

1. Read manifest of a bundle.
2. Optionally reset a disposable PostGIS schema.
3. Publish events with original event times into a replay bus namespace (replay.\* topics) or direct to services in “replay mode.”
4. Record outputs to an evidence directory.
5. Diff against golden outputs with tolerances.

Production topic pollution is a critical failure — replay namespaces and credentials must be hard-separated.

## R.2 Time acceleration

For CI, allow  $\times 10$  replay of event time while preserving relative deltas. Watermark logic must use event time, not wall clock, or accelerated replay falsely triggers LATE storms.

## R.3 Differential tolerances

Examples:

- Track position RMSE within X metres of golden at matched timestamps
- Same set of observation IDs in evidence within Jaccard threshold
- Alert tier equal under pinned policies

Do not require identical UUIDs for tracks if the tracker mints new IDs; compare on evidence fingerprints instead.

---

# Appendix S — Comparative notes on related streaming and geo systems

---

## S.1 Why not “just use a cloud IoT pipeline”

Managed IoT hubs excel at device connectivity. Counter-Swarm still needs canonicalisation, quarantine semantics, fusion-adjacent lineage, and on-prem friendliness for some customers. Cloud pub/sub can implement the log, but the **platform behaviours** in this monograph remain.

## S.2 Why not a pure time-series database as hot store

TSDB excellence in downsample and retention does not replace relational decision/audit integrity or rich spatial joins. PostGIS first remains justified; TSDB as extension or side system later is incremental.

## S.3 Why not feature stores at Stage 0

ML feature stores solve training/serving skew for model features. They are not a substitute for observation contracts. Introduce when ML engineering demonstrates training pipelines that need them — not as an ingest backbone.

---

## Appendix T — Implementation blueprints mapped to Counter-Swarm services

---

### T.1 sensor-adapter-\*

Responsibilities:

- Authenticate to vendor SDK or sim.
- Map vendor payload → candidate observation.
- Compute idempotency\_key.
- Buffer on failure; shed-oldest; export metrics.
- Never write PostGIS directly.

### T.2 normaliser

Responsibilities:

- Schema validate.
- Dedupe.
- CRS/time checks.
- Enrich IDs and receipt timestamps.
- Produce observation or quarantine.
- Emit quality metrics.

Non-responsibilities: risk scoring, alert tiering, track association.

### T.3 event-bus

Responsibilities:

- Durability, ACLs, retention, lag exposure.
- Schema registry integration.

### T.4 Warm archiver consumer

Responsibilities:

- Write Parquet partitions by site/day.
- Verify counts against bus metrics.
- Register partition locations for replay driver.

### T.5 Operator API (DE-facing endpoints)

- Get observation by ID (hot/warm).
- Get lineage from decision/track.
- Pin evidence.
- Quarantine search (RBAC).

---

## Appendix U — Threats to validity and research limits

---

This monograph’s recommendations are grounded in systems literature and the repository’s Stage 0 architecture, not in a completed multi-month field instrumentation study on live swarming UAS. External validity risks:

- Sensor rate assumptions may be low for some RF apertures.
- Customer may forbid Kafka-API components.
- Legal regimes may force shorter EO retention than technical preference.
- Fusion algorithm choices (ML-owned) may impose stricter time sync than DE MVP thresholds.

Pass 2 of the research programme should incorporate measured ingest distributions from the simulation engine and at least one hardware-in-the-loop adapter.

---

## Appendix V — Extended annotated bibliography notes

---

**Kleppmann (2017)** — Use chapters on encoding, replication, and stream processing as onboarding for engineers new to log-centric design. The book’s treatment of clocks directly supports dual timestamps and skew flags.

**Kreps et al. (2011) and Kreps (2013)** — Justify the bus as the integration spine rather than point-to-point integrations between detection, tracking, and audit.

**Akida et al. (2015)** — Provides vocabulary for late data; even consumer-side implementations should use the same words in runbooks to avoid ad-hoc synonyms (late, tardy, delayed) meaning different thresholds.

**Hall & Llinas (1997); Liggins et al. (2008); Waltz & Llinas (1990)** — Establish that registration, alignment, and pedigree are classical fusion prerequisites — i.e., data engineering is not an afterthought bolted onto “the algorithm.”

**Bar-Shalom / Blackman & Popoli** — Remind DE that association gates consume time and spatial uncertainty; schema must carry uncertainty hooks even if early trackers are simple.

**PostGIS / OGC Simple Features** — Practical geospatial interoperability baseline for operator maps and geofences.

**Corbett et al. Spanner / TrueTime (2012)** — Conceptual support for explicit time uncertainty rather than false precision.

**Lamport (1978)** — Conceptual support for not pretending a global total order across sensors exists without protocol.

**Narkhede et al. Kafka Guide** — Operational companion for producer/consumer configuration matching ICD-02/03.

**Marz & Warren (2015)** — Historical Lambda contrast; prefer log replay (Kappa-like) for this platform unless batch recomputation is specifically justified.

---

## Appendix W — Sample contract test catalogue (normative intent)

---

| Test ID    | Intent                              | Pass criterion                                   |
|------------|-------------------------------------|--------------------------------------------------|
| CT-OBS-001 | Valid minimal observation           | Schema accept; produce success                   |
| CT-OBS-002 | Missing provenance                  | Quarantine SCHEMA_INVALID or required-field code |
| CT-OBS-003 | Lat 95                              | Quarantine RANGE_LAT                             |
| CT-OBS-004 | Duplicate idempotency_key           | Single observation_id effect; dedupe metric +1   |
| CT-OBS-005 | Unknown field on spine              | Reject/quarantine (additionalProperties false)   |
| CT-OBS-006 | Unknown field in measurement        | Accept if size OK                                |
| CT-OBS-007 | Non-UTC timestamp                   | Quarantine or normalised with flag per policy    |
| CT-OBS-008 | Clock skew hard                     | Quarantine CLOCK_SKEW                            |
| CT-OBS-009 | Consumer ignores optional new field | Old consumer reads fixture v1.1                  |
| CT-TRK-001 | Evidence IDs round-trip             | Projector stores all IDs                         |
| CT-REP-001 | Bundle replay namespace isolation   | Zero writes to prod topics                       |
| CT-Q-001   | Corrupt JSON                        | Quarantine; fusion count unchanged               |

---

## Appendix X — Runbook sketches

---

### X.1 Quarantine flood

1. Identify top reason\_codes and sensor\_id.
2. If SCHEMA\_INVALID dominated by one adapter version → roll back adapter.
3. If CLOCK\_SKEW site-wide → check site time source; declare site degraded.
4. Do not disable quarantine to “keep data flowing.”

### X.2 Projection rebuild

1. Pause projector.
2. Snapshot DB.
3. Truncate projection tables or create new schema version.
4. Reconsume from warm archive or bus earliest needed offset.
5. Validate counts vs archive.
6. Switch Operator API to rebuilt schema.

## **X.3 Retention incident**

1. If bus TTL expired and warm missing → declare data loss; incident report.
  2. Improve archiver SLO; add canary.
  3. Never pretend operators can replay what was not archived.
- 
- 

## **Appendix Y — Full worked design: from vendor radar plot to operator evidence drill-down**

---

This appendix walks a single radar detection through Counter-Swarm’s data path with enough concreteness that an implementer can derive interfaces, tables, and tests without inventing a parallel architecture.

### **Y.1 Vendor plot arrives**

An ACME radar SDK callback yields a binary or JSON plot: local range-azimuth-elevation, SNR, vendor track hint, vendor sequence number, and a sensor clock timestamp. The `sensor-adapter-radar-acme` process:

1. Authenticates the SDK session using site credentials (never hard-coded in images).
2. Converts RAE to geodetic using the sensor pose from inventory (cached with version).
3. Builds a candidate observation:

```
{
  "observation_id": "to-be-assigned-by-normaliser-or-adapter",
  "schema_version": "1.0.0",
  "sensor_id": "siteA-radar-01",
  "source_type": "radar",
  "timestamp_utc": "2026-09-19T20:15:03.221Z",
  "received_at_utc": "2026-09-19T20:15:03.401Z",
  "idempotency_key": "siteA-radar-01:seq:948221",
  "location": {
    "crs": "EPSG:4326",
    "lat": 51.5074,
    "lon": -0.1278,
    "alt_m": 120.0,
    "uncertainty": {"s_major_m": 25, "s_minor_m": 10, "orient_deg": 35}
  },
  "measurement": {
    "range_m": 4200,
    "az_deg": 187.2,
    "el_deg": 4.1,
    "snr_db": 14.5,
    "vendor_track_hint": "VT-8891"
  },
  "confidence": 0.62,
  "quality": {"time_sync_mode": "gnss"},
  "provenance": {
    "adapter": "adapter-radar-acme",
    "adapter_version": "0.3.1",
    "raw_ref": "s3://csd-raw/siteA/radar/2026/09/19/plot-948221.bin"
  },
  "metadata": {"site_id": "siteA", "confidence_semantics":
"snr_proxy_calibrated_v1"}
}
```

1. Writes raw bytes to object storage when policy requires; on object-store failure, either quarantine later or buffer raw locally with the candidate (policy flag).
2. Sends candidate to normaliser with at-least-once semantics.

## Y.2 Normaliser decisions

- Schema valid → continue.
- Idempotency cache miss → continue.
- Range checks pass.
- Skew  $|received\_at - timestamp| = 180ms \rightarrow$  soft OK.
- Rate within baseline → OK.
- Assign `observation_id` if adapter left placeholder.
- Produce to `obs.observation.v1` keyed by `siteA` (partition locality).

Metrics incremented: `obs_produced_total{sensor_id=...,adapter=...}`.

## Y.3 Consumers

- **Detection** may passthrough radar plots as detections with `model_id=passthrough`.

- **Tracking** associates plot to track T-2031, emits `track.update.v1` with `evidence_obs_ids` including this observation.
- **Warm archiver** appends to day's Parquet.
- **Audit sampler** may sample 1% for integrity monitoring.

## Y.4 Hot projection

Track projector upserts:

- `tracks` row for T-2031 with geography point and state JSON.
- `track_evidence` row linking observation ID.
- Optionally upserts `observations_hot` for recent drill-down.

## Y.5 Operator drill-down

Operator selects track → API returns evidence list → user opens observation → API fetches hot row or warm Parquet by ID → optional raw download via signed URL if RBAC allows.

## Y.6 Where the design fails if short-cut

If the adapter wrote PostGIS directly and skipped the bus, warm archive and detection fan-out would require dual writes; Kleppmann's dual-write failure appears on the first partial outage. If `idempotency_key` were random UUID per retry, link flap would duplicate tracks. If `raw_ref` were omitted always, EO/radar disputes become unresolvable.

---

# Appendix Z — Contradiction handling patterns for fusion-facing DE

---

## Z.1 Types of contradiction

1. **Spatial:** two sensors claim incompatible positions for an associated object.
2. **Class:** modality classifiers disagree.
3. **Existence:** one modality detects, another with overlapping coverage does not (may be expected).
4. **Kinematic:** implied velocity from association exceeds modality physics.
5. **Identity:** vendor track hints conflict with platform track IDs.

## Z.2 Platform responses (data-shaped)

| Contradiction        | Data response                                           | Anti-pattern                 |
|----------------------|---------------------------------------------------------|------------------------------|
| Spatial incompatible | Reject association; keep separate tracks; flag          | Average positions silently   |
| Class conflict       | Distribution + CLASS_CONFLICT                           | Winner-take-all without flag |
| Existence mismatch   | Coverage-aware missing-info                             | Force delete track           |
| Kinematic impossible | Reject link; quarantine assoc event                     | Trust vendor hint blindly    |
| Identity conflict    | Prefer platform IDs; record vendor hints in measurement | Rename tracks uncontrollably |

## Z.3 DE interfaces that enable honest fusion

Fusion algorithms (ML/DS owned) need fields DE must guarantee exist when known:

- spatial uncertainty
- time uncertainty
- modality
- confidence semantics tags
- sensor pose version used for geolocation

Without those, statistical gates become magic numbers divorced from measurement reality.

---

## Appendix AA — Schema evolution case studies (hypothetical but realistic)

---

### AA.1 Adding `site_id` as a required spine field

Today `site_id` lives in metadata in examples. Making it required is a **breaking** change for producers that omit it.

Playbook:

1. Stage expand: document `metadata.site_id` as strongly recommended; metrics for missing.
2. Dual-read consumers: prefer spine `site_id` if present else metadata.
3. Deploy adapter updates.
4. Major bump to `observation.v2` or minor if still optional — only make required after 100% producer compliance for N days.
5. Contract: remove metadata fallback after deadline.

### AA.2 Changing confidence semantics

If radar adapters historically emitted SNR-proxy in confidence and a new calibrated probability arrives, do **not** overwrite meaning in place.

Playbook:

1. Add `confidence_calibrated` optional field.
2. Keep old field; document deprecation.
3. Fusion prefers calibrated when present.
4. Remove old field only in next major after consumers migrate.

### AA.3 Moving from JSON to Avro on the wire

Logical schema unchanged; encoding changes.

Playbook:

1. Registry subjects for Avro.
2. Dual-produce JSON and Avro on parallel topics or headers selecting encoding.
3. Consumers migrate.
4. Disable JSON topic after lag-free period.

---

## Appendix AB — Quantitative SLOs for data platform honesty

These are starting drafts for Systems Engineering to socialise — not silent promises.

| SLO                             | Draft target                                       | Measurement                        |
|---------------------------------|----------------------------------------------------|------------------------------------|
| Silent loss                     | < 1e-6 of canary obs missing in warm within 15m    | Canary job                         |
| Quarantine correctly labelled   | ≥ 99% of injected corrupt → SCHEMA_INVALID         | Sim fault suite                    |
| Ingest delay p95 (central)      | ≤ modality budget (e.g., 500ms radar plots)        | received_at - timestamp after sync |
| Projection freshness p95        | ≤ 2s from obs produce to track evidence visibility | Trace IDs                          |
| Warm archive completeness daily | ≥ 99.9% vs bus produce counts                      | Reconcile job                      |
| Replay namespace isolation      | 0 prod topic writes in replay CI                   | ACL deny + test                    |

If SLOs cannot be measured, they are aspirations. Instrument first.

---

## Appendix AC — Interaction with simulation and digital twin data

Simulation is not a second platform. ICD-15 says sim emits vendor-shaped or canonical-candidate payloads into sim adapters. DE implications:

1. Same normaliser binary for sim and live (config differs).

2. `source_type=sim` always set for synthetic; never launder sim as radar in shared buses used for live ops.
3. Fault injectors must be configuration-driven and audited.
4. Deterministic seeds recorded in scenario manifests for CI replay.
5. Performance tests use sim at multiples of expected live rate.

A separate “sim-only shortcut schema” is technical debt that guarantees live cutover pain.

---

## **Appendix AD — Data engineering ethics and defensive purpose**

---

Building high-integrity sensing pipelines for defensive decision-support still requires ethical clarity:

- Minimise collection and retention of irrelevant personal imagery.
- Do not build covert surveillance features beyond declared site defence missions.
- Keep human approval authoritative for response categories.
- Make uncertainty visible; do not ship dashboards that perform certainty theatre.

Data engineering choices — retention, logging, ACLs, quarantine — are ethical controls with technical implementations.

---

## **Appendix AE — Curriculum: how a new DE hire should learn this platform**

---

Week 1: Read Kleppmann (selected chapters), Kreps’ log essay, `architecture/data-flow.md`, `observation.v1.json`.

Week 2: Run lab bus + normaliser; break schema on purpose; watch quarantine.

Week 3: Implement one extension schema + contract tests.

Week 4: Build a replay bundle from a sim scenario; present lineage of a decision.

Week 5: Own a soak test and an SLO dashboard.

This curriculum is deliberate: tools second, honesty first.

---

## **Appendix AF — Open research questions (honest backlog)**

---

1. What watermark strategies minimise false LATE flags across radar+EO with systematically different delays?
2. Can feasibility envelopes be learned per sensor without encoding policy into ingest?
3. What compression and columnar layouts minimise warm replay cost for mixed modalities?

4. How should confidence calibration be stored so DS can improve it without breaking DE contracts?
5. What is the minimal lineage graph that satisfies SG audits without drowning storage?

These questions belong in Pass 2 empirical work with the simulation engine — not in invented citations.

---

## Document control

---

| Field     | Value                                                |
|-----------|------------------------------------------------------|
| Author    | Victor.I                                             |
| Status    | Pass 1 structured research monograph                 |
| Integrity | Landmark citations only; no fabricated DOIs          |
| Companion | docs/data/architecture.md, architecture/data-flow.md |

**End of monograph.**